Gemini 3.1 Pro

Yes! If your goal is to build "everything media in one place," dealing with hundreds of fragmented Fandom wikis is going to be a massive headache.

There is a "holy grail" alternative that is **completely free, significantly easier to use, and infinitely better** for a cross-media app like yours: **Wikidata combined with the Wikipedia REST API.**

This combination acts as a universal translator for all your existing APIs (TMDb, OpenLibrary, Last.fm) and delivers incredibly clean summaries, lore, and metadata. Even better, Wikimedia APIs support Cross-Origin Resource Sharing (CORS) out of the box, meaning **you can call them directly from your GitHub Pages frontend** without needing to route them through your Supabase Edge Functions.

Here is exactly how this alternate stack works and why it's perfect for Catalogd.

The Wikidata "Universal Translator"

Wikidata is the massive, structured database that powers Wikipedia. Its superpower is that it stores **External Identifiers**. It already knows exactly how your existing APIs connect to each other.

Instead of searching by text (which is prone to errors), you query Wikidata using the IDs you already have.

Every external API has a specific "Property ID" in Wikidata:

- **TMDb Movie ID:** `P4947`
- **TMDb TV Series ID:** `P4983`
- **Open Library Subject ID:** `P3847`
- **Last.fm Artist ID:** `P3192`

How the Flow Works in Your App

Step 1: The User Opens a Media Page Let's say a user clicks on the movie *Dune* in your app. Your app already knows the TMDb ID for *Dune* is `438631`.

Step 2: Query Wikidata (from GitHub Pages) You make a quick, direct API call to Wikidata asking: *"What is the Wikipedia page title for the item that has the TMDb Movie ID of 438631?"*

Step 3: Fetch the Lore from Wikipedia Wikidata responds with the title: `"Dune_(2021_film)"`. Now, you use the **Wikipedia REST API**, which has a beautiful endpoint specifically designed for modern apps called `/page/summary/`.

You make a call to:
https://en.wikipedia.org/api/rest_v1/page/summary/Dune_(2021_film)

Step 4: Display Clean JSON Unlike Fandom or standard MediaWiki APIs which return messy wiki-text that you have to parse, the Wikipedia REST API returns a gorgeous, clean JSON object containing:

- `extract` : A perfect, plain-text summary of the media or character.
- `extract_html` : A pre-formatted HTML version if you prefer.
- `thumbnail` : A high-quality main image URL.

Why This Beats Fandom for Your Stack

Feature	Fandom API	Wikidata + Wikipedia
Endpoint	Fragmented (thousands of custom URLs)	**Single Global Endpoint**
CORS Support	Often blocked (Requires Edge Functions)	**Allowed** (Direct from GitHub Pages)
Matching Method	Messy text search / Scraping	**Exact ID matching** (TMDb, OpenLibrary)
Data Format	Clunky Wikitext	**Clean JSON summaries**
Media Coverage	Heavily skews toward sci-fi/fantasy	**Truly Universal** (Books, indie music, old films)

How to Supercharge Your App with This

1. **Direct Frontend Fetching:** Because you don't have to bounce through Supabase Edge Functions to bypass CORS, your page load speeds will be significantly faster.
2. **AI Recommendations (Qdrant):** You can still feed the rich text summaries you pull from Wikipedia directly into Qdrant. Because Wikipedia covers historical context, thematic elements, and production details, your AI embeddings will be incredibly rich.
3. **The "Six Degrees" Feature:** Because Wikidata connects everything, you can easily build features like: *"Show me other movies based on books by this OpenLibrary author that feature soundtracks by this Last.fm artist."*